

AI Laboratory — Bayesian Networks and Autoregressive Language Models

Atharva Naik

October 1, 2026

Part I: From Probability to Language

Question 1 — Why is this decomposition useful for generating text?

The chain-rule decomposition $P(X_1, \dots, X_T) = P(X_1) \prod_{t=2}^T P(X_t \mid X_1, \dots, X_{t-1})$ turns one intractable joint-probability object (a distribution over every possible sequence of T tokens, which has exponentially many outcomes) into a product of much simpler, smaller conditional distributions — each one just “what comes next, given what came before.” This is useful for *generating* text specifically because generation can then proceed one token at a time: sample X_1 , then sample X_2 from $P(X_2 \mid X_1)$, then X_3 from $P(X_3 \mid X_1, X_2)$, and so on. We never need to represent or sample from the full joint distribution directly — we only ever need a model of “next token given context,” which is both tractable to estimate from data and natural to apply sequentially. This is exactly what an autoregressive language model does.

Part II: A Bayesian Network for Text

Question 2 — What independence assumption is being made by this network?

The network $X_1 \rightarrow X_2 \rightarrow X_3 \rightarrow X_4$ encodes a **first-order Markov assumption**: each token is conditionally independent of all tokens before its immediate predecessor, given that immediate predecessor. In probability notation:

$$P(X_t \mid X_1, \dots, X_{t-1}) = P(X_t \mid X_{t-1}) \quad \text{for all } t.$$

Equivalently, $X_t \perp \{X_1, \dots, X_{t-2}\} \mid X_{t-1}$ — once we know the immediately preceding word, knowing any earlier words gives no additional information about the next word.

Part III: Build a Small Language Dataset

Using the lab’s starting dataset, lower-cased, tokenised on whitespace, and wrapped with <START>/<END>:

```
<START> the cat sat on the mat <END>
<START> the cat sat on the rug <END>
<START> the dog sat on the mat <END>
<START> the dog ran to the park <END>
<START> the cat ran to the park <END>
<START> the dog sat on the rug <END>
```

(dataset.py, attached.)

Part IV: Constructing the Conditional Probability Table

Question 3 — Construct $P(\text{next word} \mid \text{current word})$ for at least the, cat, dog, sat, ran. Identify any zero-probability transitions.

Computed directly from transition counts over the dataset above:

Context	Next word	Count	Probability
the	cat	3	$3/12 = 0.2500$
the	dog	3	$3/12 = 0.2500$
the	mat	2	$2/12 = 0.1667$
the	park	2	$2/12 = 0.1667$
the	rug	2	$2/12 = 0.1667$
cat	sat	2	$2/3 = 0.6667$
cat	ran	1	$1/3 = 0.3333$
dog	sat	2	$2/3 = 0.6667$
dog	ran	1	$1/3 = 0.3333$
sat	on	4	$4/4 = 1.0000$
ran	to	2	$2/2 = 1.0000$

Zero-probability transitions. Every word pair not listed above has $C(w_i, w_j) = 0$ and hence $P(w_j \mid w_i) = 0$. Notable examples: $P(\text{dog} \mid \text{cat}) = 0$, $P(\text{cat} \mid \text{dog}) = 0$ (the two animal nouns never follow each other directly), $P(\text{mat} \mid \text{cat}) = 0$ (an object noun never immediately follows a subject noun — “the” always intervenes), $P(\text{the} \mid \text{sat}) = 0$, and $P(w \mid \text{sat}) = 0$ for every $w \neq \text{on}$, since sat is followed by on with probability 1 in every training example (likewise ran→to). This sparsity is a direct consequence of the tiny training set — most of the 12×12 possible word-pairs were simply never observed.

Part V: Ask an LLM to Implement the Model

Exact prompt used (the lab’s suggested prompt, verbatim):

“Write a simple Python implementation of a first-order autoregressive language model. The model should: 1. take a list of tokenised sentences as training data; 2. count transitions between consecutive tokens; 3. construct the conditional distribution $P(X_t \mid X_{t-1})$; 4. display the probabilities for a specified previous token; 5. predict the most probable next token; 6. generate a sentence by repeatedly sampling the next token; 7. stop when the <END> token is generated. Do not use a machine-learning library or a pretrained language model. Use ordinary Python data structures and random sampling.”

(`first_order_model.py`, attached — see Part VI below for corrections made to the generated code before accepting it.)

Part VI: Inspect the LLM-Generated Code

Question 4 — Where are the transition counts stored? In `self.transition_counts`, a `defaultdict(Counter)` built in `_count_transitions`: `transition_counts[w_i][w_j]` holds $C(w_i, w_j)$, the number of times token w_j immediately followed token w_i anywhere in the training sentences.

Question 5 — Where is $P(X_t \mid X_{t-1})$ computed? In `_build_cpt`, called once from `__init__` and stored in `self.probabilities`: for each previous-token key `w_i`, it divides every count `transition_counts[w_i][w_j]` by `total = sum(counter.values())`, giving `probabilities[w_i][w_j] = C(w_i, w_j) / sum_k C(w_i, w_k)` — exactly the formula from Part IV.

Question 6 — How does the program choose the next word? Always most probable, or sampling? Explain the difference. The implementation supports **both**, as separate methods: `predict_most_probable` always returns $\arg \max_w P(w \mid \text{prev})$ (deterministic — the same context always produces the same prediction), while `sample_next` uses `rng.choices(words, weights=probs, k=1)` to draw a word **randomly, in proportion to its probability** (so a word with probability 0.25 is chosen roughly a quarter of the time, not always or never). `generate_sentence(mode=...)` picks which one to use at each step. The difference matters for generation: `always-argmax` (greedy) gives a single deterministic continuation per context, while `sampling` gives varied output across runs — explored concretely in Parts IX/X below.

Question 7 — What happens if the program encounters a word for which no transition has been observed? `next_token_distribution` returns an **empty dict** `{}` rather than raising a `KeyError` (this was one of the corrections made to the first draft — see below). Both `predict_most_probable` and `sample_next` check for this and return `None`, and `generate_sentence` treats `None` as a signal to stop generation early. This means an unseen context cleanly halts generation rather than crashing or silently fabricating a transition.

Corrections made to the LLM-generated code before accepting it:

1. The first draft's `predict_most_probable` broke probability ties by whichever key Python's dict iteration happened to return first (insertion order), which is not reproducible or easily explainable; corrected to break ties alphabetically.
2. The first draft risked a `KeyError` for an unseen previous token (directly relevant to Question 7); corrected `next_token_distribution` to return `{}` explicitly, with callers checking for it.
3. Added a `random.Random` parameter to `generate_sentence` for reproducible generated-text examples in this report.

Part VII: Test the Probability Model

Testing the invariant $\sum_v P(v \mid w) = 1$ for every observed context w (`test_normalisation.py`):

word	$\sum_v P(v \mid w)$	OK?
<START>	1.0000000000	OK
the	1.0000000000	OK
cat	1.0000000000	OK
dog	1.0000000000	OK
sat	1.0000000000	OK
ran	1.0000000000	OK
on	1.0000000000	OK
to	1.0000000000	OK
mat	1.0000000000	OK
rug	1.0000000000	OK
park	1.0000000000	OK

All 11 observed contexts normalise to 1.0.

Question 8 — If one of the totals is 0.87, what does this tell you about the implementation? A total of 0.87 (rather than 1.0) means the implementation is **not correctly normalising** — something is mathematically wrong, not just numerically imprecise (floating-point error would show up as something like 0.999999997, not a 13-point shortfall). Likely causes: dividing by the wrong denominator (e.g. dividing by the total count over *all* contexts rather than just this one context’s total), losing some counts before normalising (e.g. an off-by-one in the counting loop, or accidentally skipping some sentences), or filtering out some continuations after counting but before computing probabilities without renormalising the remainder. This is exactly the kind of bug that *passing* unit tests on “does it run” would miss, but that the explicit normalisation test catches — illustrating why Part VII’s check is a real correctness test, not a formality.

Part VIII: Predicting the Next Word

Next-word distributions and $\arg \max_w P(w \mid \text{context})$ for six contexts:

$P(X_{t+1} \mid X_t = \text{the})$: cat 0.2500, dog 0.2500, mat 0.1667, park 0.1667, rug 0.1667 $\rightarrow$
 $\arg \max = \text{cat}$ (tie with dog broken alphabetically)

$P(X_{t+1} \mid X_t = \text{cat})$: sat 0.6667, ran 0.3333 $\rightarrow \arg \max = \text{sat}$

$P(X_{t+1} \mid X_t = \text{dog})$: sat 0.6667, ran 0.3333 $\rightarrow \arg \max = \text{sat}$

$P(X_{t+1} \mid X_t = \text{sat})$: on 1.0000 $\rightarrow \arg \max = \text{on}$

$P(X_{t+1} \mid X_t = \text{ran})$: to 1.0000 $\rightarrow \arg \max = \text{to}$

$P(X_{t+1} \mid X_t = \langle \text{START} \rangle)$: the 1.0000 $\rightarrow \arg \max = \text{the}$

Question 9 — Are the most probable predictions always the words you’d personally expect? What does this tell you about the difference between a probability model and human linguistic expectations? Mostly yes for the deterministic contexts (sat $\rightarrow$ on, ran $\rightarrow$ to, $\langle \text{START} \rangle \rightarrow$ the) since the tiny dataset leaves no alternative. But the the $\rightarrow$ cat prediction is more revealing: cat wins only because of an **arbitrary alphabetical tie-break** against dog (both genuinely occurred 3/12 times) — a human asked “what word most likely follows ‘the’ here?” would correctly say “cat and dog are equally likely,” not single out “cat.” This exposes a real gap: the model’s probabilities are a faithful reflection of **this specific tiny training corpus’s statistics**, not of general English usage or semantic plausibility — park, mat, and rug are judged *less* likely than cat/dog here purely because of how many example sentences happened to be written down, not because of any linguistic property of the words themselves. Human expectations draw on a lifetime of language exposure and world knowledge; this model only knows six sentences.

Part IX: Generate Text

Twenty sentences generated by repeated sampling from $P(X_t \mid X_{t-1})$ (seed 42, generate.py; raw token sequences saved to generated_sentences.txt, attached):

1. the cat sat on the dog ran to the cat sat on the cat sat on the dog ran to (hit max length)
2. the dog sat on the mat
3. the park
4. the dog sat on the rug
5. the park
6. the cat sat on the cat sat on the mat
7. the mat
8. the dog sat on the mat
9. the rug
10. the dog sat on the mat
11. the park
12. the mat
13. the mat
14. the mat
15. the mat
16. the rug
17. the cat sat on the cat sat on the park
18. the dog ran to the dog sat on the park
19. the rug
20. the park

Most sampled sentences are short and grammatical (the dog sat on the mat), but a first-order model has no memory of the whole sentence — only the immediately preceding word — so it can loop back into the after mat/rug/park/on and produce run-on, repetitive sequences like sentence 1 or 6 that a real sentence boundary would never allow.

Part X: Deterministic vs Probabilistic Generation

Mode A — greedy ($\arg \max_w P(w \mid w_{\text{previous}})$ every step), 5 sentences:

- 1-5. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on (identical, a

Mode B — sampling, 5 sentences:

1. the cat sat on the mat
2. the cat sat on the rug
3. the cat sat on the mat
4. the park
5. the cat sat on the rug

Distinct sentences: **greedy 1/5, sampling 3/5.**

Question 10 — Compare the two sets. Which mode produces more variation? Why? Sampling produces far more variation (3 distinct sentences out of 5,

versus greedy’s 1 out of 5 — literally the same sentence every single time). The reason is structural, not incidental: tracing the greedy argmax chain reveals a **cycle** — $\arg \max P(\cdot \mid \text{the}) = \text{cat}$, $\arg \max P(\cdot \mid \text{cat}) = \text{sat}$, $\arg \max P(\cdot \mid \text{sat}) = \text{on}$, $\arg \max P(\cdot \mid \text{on}) = \text{the}$ — so greedy decoding loops $\text{the} \rightarrow \text{cat} \rightarrow \text{sat} \rightarrow \text{on} \rightarrow \text{the} \rightarrow \text{cat} \rightarrow \text{sat} \rightarrow \text{on} \rightarrow \dots$ forever and never reaches $\langle \text{END} \rangle$ (generation only stops here because of the `max_len=20` safety cap, not because the model naturally terminated). Greedy decoding is a **deterministic function of the context**, so identical starting conditions always produce identical output; sampling injects randomness at every step in proportion to the learned probabilities, so different runs genuinely explore different paths through the distribution — including the lower-probability but real continuations (`park`, `rug`) that greedy decoding can never reach because they are never anyone’s single most-probable choice.

Part XI: A Second-Order Bayesian Network

Question 11 — How does the second-order model differ from the first-order model in terms of (1) graph structure, (2) the conditional probability table, (3) context available for prediction, (4) data needed?

1. **Graph structure.** The first-order network is a simple chain $X_1 \rightarrow X_2 \rightarrow X_3 \rightarrow \dots$, where each node has exactly one parent. The second-order network has each X_t receiving **two** incoming edges, $X_{t-2} \rightarrow X_t \leftarrow X_{t-1}$, so each node (from X_3 onward) has two parents instead of one.
2. **Conditional probability table.** The first-order CPT is indexed by a single previous word, $P(X_t | X_{t-1})$ — one row per vocabulary word. The second-order CPT is indexed by an *ordered pair* of previous words, $P(X_t | X_{t-2}, X_{t-1})$ — one row per possible word-*pair*, so the table has (up to) $|V|$ times more rows for the same vocabulary V .
3. **Context available for prediction.** The second-order model can condition on two tokens of history instead of one, letting it capture short-range dependencies the first-order model structurally cannot see (e.g. distinguishing cat sat from dog sat as different contexts, rather than collapsing both down to just “sat”).
4. **Data needed.** Because the context space is roughly $|V|$ times larger, far more training data is needed to observe enough examples of each specific *pair* of preceding words — our second-order CPT ended up observing only 14 of the $12^2 = 144$ possible 2-word contexts (Part XIII), versus 11 of 12 possible 1-word contexts for the first-order model, despite using the exact same six training sentences.

Part XII: Use the LLM Again

Prompt used (the lab’s suggested prompt, verbatim):

“Modify the existing first-order autoregressive model into a second-order model. The model should estimate $P(X_t | X_{t-2}, X_{t-1})$. Represent the model using counts of observed triples and use these counts to construct conditional probability distributions. Do not replace the model with a neural network or a pretrained language model.”

Corrections made to the generated code before accepting it (`second_order_model.py`, attached):

1. The first draft keyed contexts by a Python **list** `[w_{t-2}, w_{t-1}]`, which is unhashable and cannot be used as a dict key; every context was changed to a **tuple** `(w_{t-2}, w_{t-1})`.
2. The first draft didn’t handle generation at $t = 2$, where only one previous token (`<START>`) exists and no genuine two-token context is yet available; corrected to fall back to the first-order model’s $P(X_2 | X_1)$ for exactly that first step, then switch to the true second-order model from $t = 3$ onward — matching the chain-rule example on the lab’s own page 2, where $P(X_1 | \langle \text{START} \rangle)$ and $P(X_2 | X_1)$ remain lower-order terms even within a higher-order factorisation.

3. Same alphabetical tie-breaking and “unseen context $\rightarrow \{\}/\text{None}$ ” handling as the first-order model, for consistency.

Selected second-order CPT rows (`second_order_model.py`’s `__main__`, also deliverable item 3):

Context (X_{t-2}, X_{t-1})	$P(X_t \mid \cdot)$
(<START>, the)	cat 0.5000, dog 0.5000
(the, cat)	sat 0.6667, ran 0.3333
(the, dog)	sat 0.6667, ran 0.3333
(cat, sat)	on 1.0000
(dog, sat)	on 1.0000
(sat, on)	the 1.0000

The normalisation test from Part VII was re-run on this model (same property, $\sum_v P(v \mid \text{ctx}) = 1$): **all 14 observed contexts normalise correctly to 1.0** (verified, not just asserted — full output in the appendix script `second_order_model.py`).

Part XIII: Comparing the Two Models

(compare.py, attached.)

Measure	First-order	Second-order
Observed contexts	11 (of 12 possible)	14 (of 144 possible)
Table cells (raw)	17	18
Free parameters	6	4
Deterministic (zero-entropy) contexts	8 / 11	10 / 14
Unseen possible contexts	1 / 12	130 / 144
Distinct sentences (50 samples)	25 / 50	6 / 50

Qualitative examples (5 sampled sentences each):

First-order: the park / the rug / the dog sat on the rug / the rug / the park

Second-order: the dog ran to the park / the cat sat on the rug / the dog sat on the mat / the dog sat on the mat / the cat sat on the mat

The second-order samples are **qualitatively more coherent** — every one of the five is a complete, grammatical sentence that actually appears (or near-matches) the training data, because a context like (cat, sat) deterministically forces on next, chaining reliably to a well-formed sentence. The first-order samples are shorter and sometimes trail off as a bare object (the park, the rug) because the model only remembers the was last seen and has no memory of whether a verb phrase is “in progress.” But this coherence comes at a steep cost in **diversity**: only 6 distinct second-order sentences emerged out of 50 samples (it mostly just reconstructs the 6 training sentences verbatim) versus 25 distinct first-order sentences — the second-order model’s narrower, more deterministic contexts leave little room to recombine words in new ways.

Question 12 — Why does increasing context potentially improve prediction, but simultaneously make the model harder to estimate from limited data? Relate your answer to the size of the CPT. More context lets the model distinguish situations that look identical with less context but actually call for different continuations — e.g. first-order collapses “cat sat” and “dog sat” into the same context (sat), while second-order can in principle treat them differently, which is strictly more expressive and can produce more accurate, more coherent predictions. But the CPT’s **size grows combinatorially with context length**: a first-order CPT has at most $|V|$ rows (one per single word), while an order- k CPT has at most $|V|^k$ rows (one per k -word combination) — here 12 versus 144, a $12\times$ blowup just going from order 1 to order 2. Since the amount of *training data* didn’t grow at all (still the same six sentences), each row gets observed far fewer times on average — most of the 144 possible 2-word contexts (130 of them, 90%) were never observed at all, versus only 1 of 12 for the first-order model. This is the classic **bias-variance / data-sparsity tradeoff**: more context reduces bias (the model can represent finer-grained dependencies) but increases variance (each specific context is estimated from far fewer — often zero — examples), exactly as observed in the parameter-count and coverage numbers above.

Part XIV: The Connection to Modern Language Models

A modern autoregressive LM models the same object, $P(X_t \mid X_1, \dots, X_{t-1})$, factorised the same way, $P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1})$ with $P(x_1 \mid \langle \text{START} \rangle)$ as the first term. The difference is entirely in *how the conditional distribution is represented and learned*: this lab’s models use small, explicit, hand-countable conditional probability tables; a modern LM instead trains a neural network (with learned weights, via gradient-based optimisation on huge corpora) to estimate the same conditional distribution over a much larger context window and vocabulary. The underlying probabilistic question, “predict the next token conditional on previous tokens,” stays identical across both; only the engineering machinery used to answer it changes.

Part XV: Reflection on the Role of the LLM

Question 13 — Why is Approach B (“Implement $P(X_t \mid X_{t-1})$, estimated from transition counts, with sampling-based generation”) preferable to Approach A (“Write a Python language model for me”) when constructing an intelligent system?

- **Specifying the intended behaviour.** Approach B states exactly what probabilistic object is being computed ($P(X_t \mid X_{t-1})$), how it’s estimated (transition counts), and how generation works (sampling) — there is one unambiguous correct implementation to check against. Approach A leaves the LLM to guess the architecture, the estimation method, and the generation strategy, so “correct” isn’t even well-defined; two different LLM runs could produce two incompatible but both “reasonable-looking” programs.
- **Understanding the representation.** Writing Approach B’s prompt requires the person to already know it should be a CPT built from counts, not (say) a neural net or a lookup of pretrained embeddings — which forces understanding of the model *before* delegating the typing to an LLM, exactly the lab’s “Understand → Design → Ask the LLM” workflow (Section 3).
- **Validating the generated implementation.** Because Approach B’s spec is precise, the resulting code can be checked line-by-line against it (Part VI’s Questions 4–7) — “is this where the counts are stored? Is this where $P(X_t \mid X_{t-1})$ is computed?” Approach A’s vague spec gives no such checklist to validate against.
- **Testing probabilistic invariants.** Approach B’s explicit probabilistic framing makes it obvious that $\sum_v P(v \mid w) = 1$ must hold (Part VII) — a concrete, checkable property. A vaguely-specified “language model” from Approach A might not even expose its internal probabilities in a way that’s testable at all.
- **Distinguishing implementation from model.** Approach B keeps the *probabilistic model* (what should be computed) and the *implementation* (the code that computes it) conceptually separate, so a bug in the code can be identified as a bug (an implementation failing to match the model) rather than mistaken for “the model is just like that.” Approach A conflates the two — without a stated model, there’s no independent standard the implementation could fail to meet.

In short: Approach B treats the LLM as **a tool for constructing a system whose behaviour was already decided**, while Approach A asks the LLM to make the de-

sign decisions too, which removes the very thing (a precise specification) that makes verification possible — directly echoing Section 3’s framing that “the LLM is a tool for constructing the intelligent system, not a replacement for understanding the system.”

Final Question: What Did the Bayesian Network Add?

Question 14 — What did thinking of the language model as a Bayesian network give you? Discuss at least three of the listed benefits.

1. **A representation of dependencies.** Drawing $X_{t-1} \rightarrow X_t$ (or $X_{t-2}, X_{t-1} \rightarrow X_t$) made explicit, visually and structurally, exactly which variables the model assumes the next word depends on — and, just as importantly, which it assumes it does *not* depend on (everything before X_{t-1} , in the first-order case).
2. **A factorisation of the joint distribution.** Instead of needing to specify or estimate one enormous joint distribution over entire sentences, the network structure directly handed us the chain-rule factorisation into small, individually-estimable conditional terms — this is what made “count transitions, then divide” a tractable estimation procedure at all.
3. **A principled method for generation.** The network’s generative story — sample X_1 , then $X_2 \sim P(X_2 | X_1)$, then $X_3 \sim P(X_3 | X_2)$, and so on — translated directly into the `generate_sentence` sampling loop; the Bayesian network wasn’t just a diagram, it was literally the generation algorithm.
4. **A way to reason about independence assumptions.** Comparing the first-order chain to the second-order fork made the cost of the independence assumption concrete and measurable (Part XIII): going from “assume $X_t \perp X_1, \dots, X_{t-2} | X_{t-1}$ ” to “assume $X_t \perp X_1, \dots, X_{t-3} | X_{t-2}, X_{t-1}$ ” directly predicted and explained the parameter-count blow-up and the resulting sparsity.
5. **A way to understand the effect of increasing context.** The BN framing made it clear *a priori* that adding a parent to each node (more context) must multiply, not add, the size of the conditional probability table — which is exactly what Part XIII’s 12→144 context-count jump demonstrated empirically.
6. **A way to test whether an implementation matches its probabilistic specification.** Because the Bayesian network specifies a precise mathematical object ($P(X_t | \text{parents}(X_t))$, with $\sum_v P(v | \text{parents}) = 1$), it gave a concrete, checkable target — which is exactly what made Part VII’s normalisation test (and Question 8’s diagnosis of a hypothetical 0.87 bug) possible in the first place.

Deliverables

1. **Python implementation of the first-order model:** `first_order_model.py` (plus `dataset.py`).
2. **Python implementation of the second-order model:** `second_order_model.py`.
3. **Conditional probability tables for selected contexts:** Part IV (first-order, by hand) and Part XII (second-order) above; full tables reproducible by running `first_order_model.py / second_order_model.py` directly.
4. **Examples of generated text:** Part IX (20 sampled sentences; raw tokens in `generated_sentences.txt`) and Part X (greedy vs. sampling, 5 each).
5. **Results of probability-normalisation tests:** Part VII (first-order, all 11 contexts) and the re-run noted in Part XII (second-order, all 14 contexts); script: `test_normalisation.py`.

6. **Answers to Questions 1-14:** throughout this document, inline with each Part.
7. **Reflection on how the LLM was used and how its output was validated:** Part XV (Question 13) discusses this directly; concretely, each generated file above lists the exact prompt used and the specific corrections made before acceptance (e.g. `second_order_model.py`'s unhashable-list-key bug, fixed by switching to tuple keys — a piece of LLM-generated code that was inspected, found incorrect, and corrected, as the deliverables ask for explicitly).

Attached code: `dataset.py`, `first_order_model.py`, `second_order_model.py`, `test_normalisation.py`, `generate.py`, `compare.py`, `generated_sentences.txt`.